Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 1.5

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 20/09/2026

Gráficos en Colaboratory

Un uso común de los cuadernos es la visualización de datos mediante gráficos. Colaboratory lo hace fácil con varias herramientas de gráficos disponibles como importaciones de Python.

Matplotlib

Matplotlib es el paquete de gráficos más común, vea su documentación para más detalles, y sus ejemplos para que te des algunas ideas.

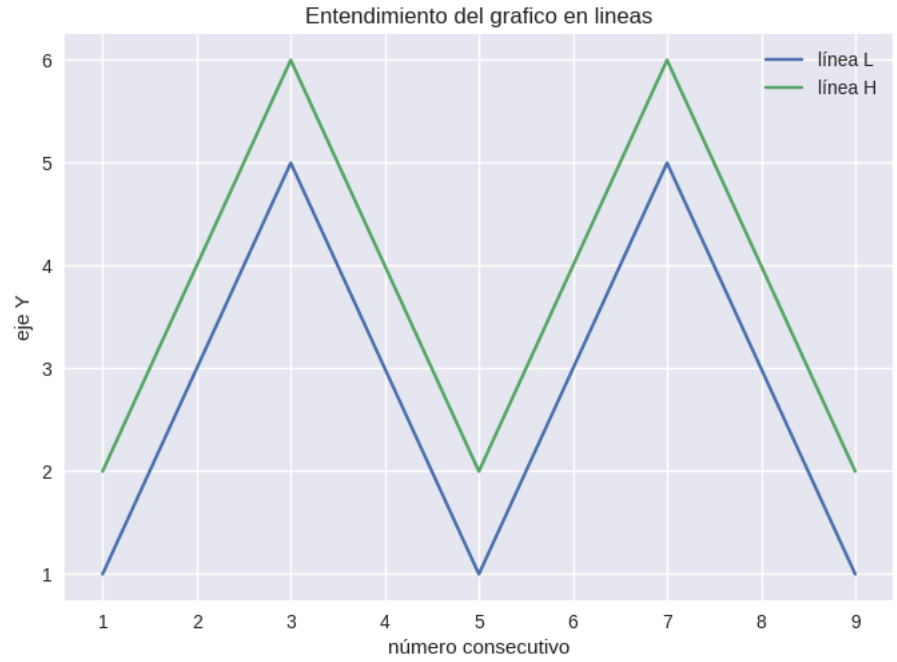
Line Plots

Gráficos de línea de puntos

```
import matplotlib.pyplot as plt

x  = [1, 2, 3, 4, 5, 6, 7, 8, 9]
y1 = [1, 3, 5, 3, 1, 3, 5, 3, 1]
y2 = [2, 4, 6, 4, 2, 4, 6, 4, 2]
plt.plot(x, y1, label="línea L")
plt.plot(x, y2, label="línea H")
plt.plot()

plt.xlabel("número consecutivo")
plt.ylabel("eje Y")
plt.title("Entendimiento del grafico en lineas")
plt.legend()
plt.show()
```



Bar Plots

Gráfico de barras

```
import matplotlib.pyplot as plt

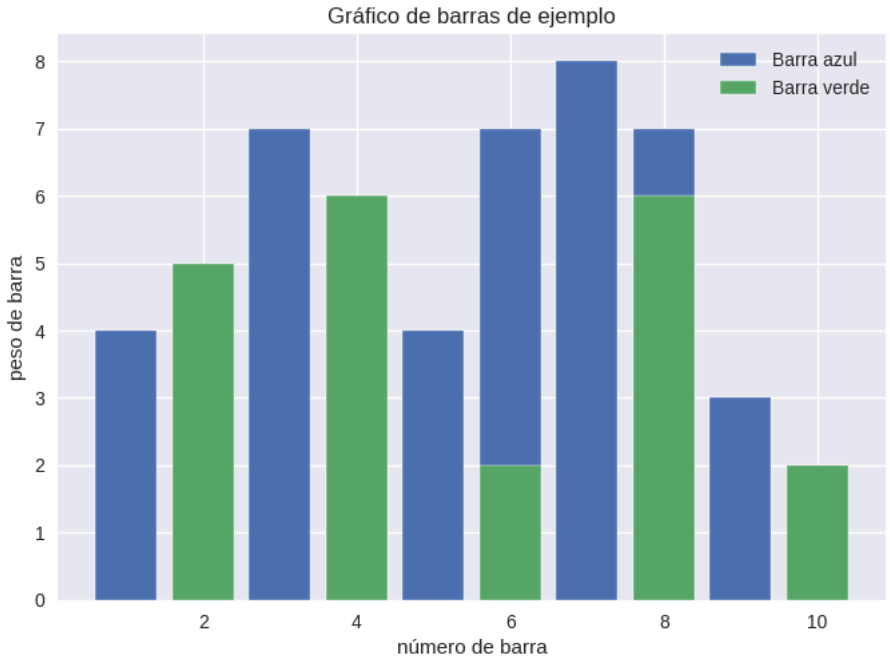
# Mira el índice 4 y  5, muestra casos superpuestos.
x1 = [1, 3, 4, 5, 6, 7,8, 9]
y1 = [4, 7, 2, 4, 7, 8,7, 3]

x2 = [2, 4, 6, 8, 10]
y2 = [5, 6, 2, 6, 2]

# Colores: https://matplotlib.org/api/colors_api.html

plt.bar(x1, y1, label="Barra azul", color='b')
plt.bar(x2, y2, label="Barra verde", color='g')
plt.plot()

plt.xlabel("número de barra")
plt.ylabel("peso de barra")
plt.title("Gráfico de barras de ejemplo")
plt.legend()
plt.show()
```



Histogramas

Histogramas de frecuencia

```
import matplotlib.pyplot as plt
import numpy as np

# Se usa numpy para genera un montón de datos al azar en una curva de campaña alrededor de 5.
n = 5 + np.random.randn(1000)

m = [m for m in range(len(n))]
plt.bar(m, n)
plt.title("Datos crudos")
plt.show()

plt.hist(n, bins=20)
plt.title("Histograma")
plt.show()

plt.hist(n, cumulative=True, bins=20)
plt.title("Histograma acumulado")
plt.show()
```

Gráfico de dispersión (Scatter Plots)

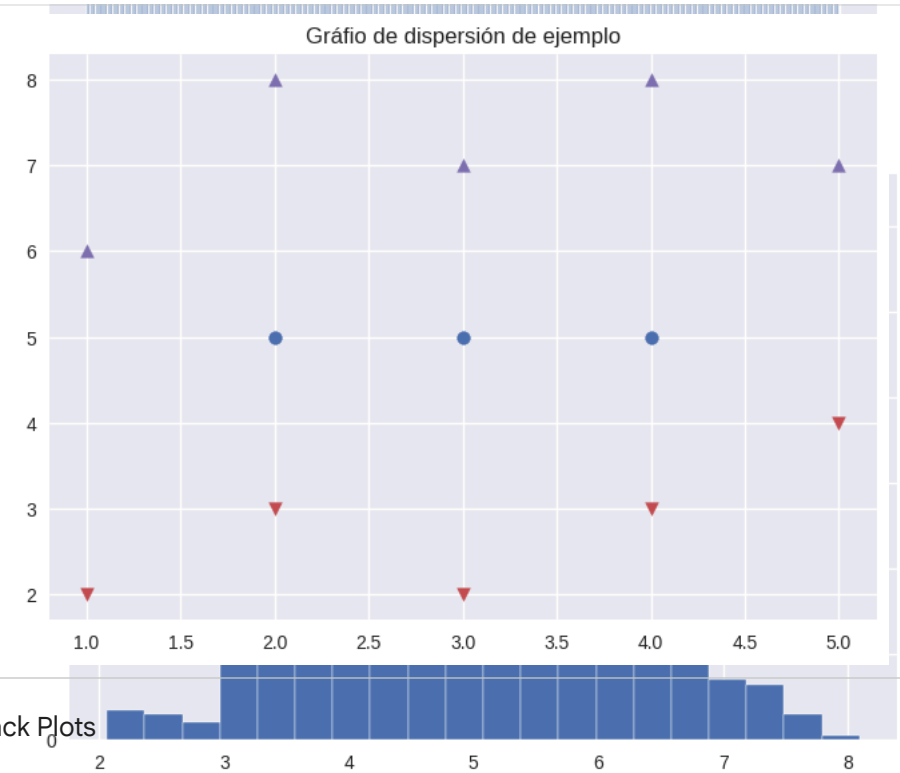
```
import matplotlib.pyplot as plt

x1 = [2, 3, 4]
y1 = [5, 5, 5]

x2 = [1, 2, 3, 4, 5]
y2 = [2, 3, 2, 3, 4]
y3 = [6, 8, 7, 8, 7]

# Markers: https://matplotlib.org/api/markers_api.html

plt.scatter(x1, y1)
plt.scatter(x2, y2, marker='v', color='r')
plt.scatter(x2, y3, marker='^', color='m')
plt.title('Gráfio de dispersión de ejemplo')
plt.show()
```



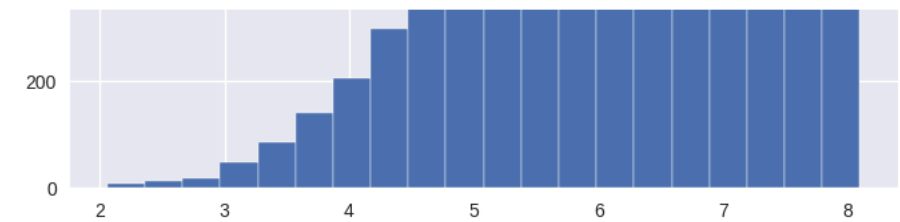
Stack Plots

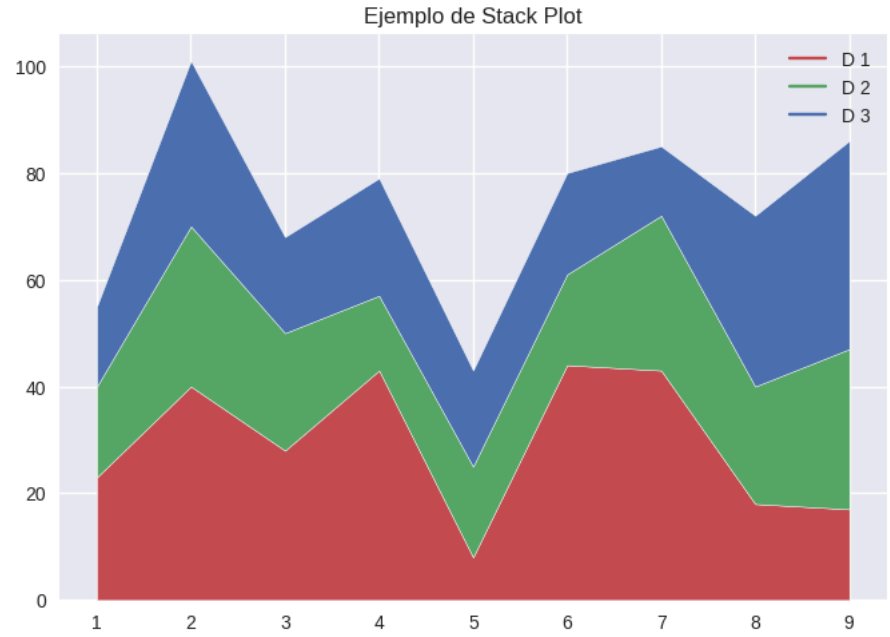
```
import matplotlib.pyplot as plt

idxes = [ 1,  2,  3,  4,  5,  6,  7,  8,  9]
arr1  = [23, 40, 28, 43,  8, 44, 43, 18, 17]
arr2  = [17, 30, 22, 14, 17, 17, 29, 22, 30]
arr3  = [15, 31, 18, 22, 18, 19, 13, 32, 39]

# Añadir una leyenda a los stack plots es complicado.
plt.plot([], [], color='r', label = 'D 1')
plt.plot([], [], color='g', label = 'D 2')
plt.plot([], [], color='b', label = 'D 3')

plt.stackplot(idxes, arr1, arr2, arr3, colors= ['r', 'g', 'b'])
plt.title('Ejemplo de Stack Plot')
plt.legend()
plt.show()
```





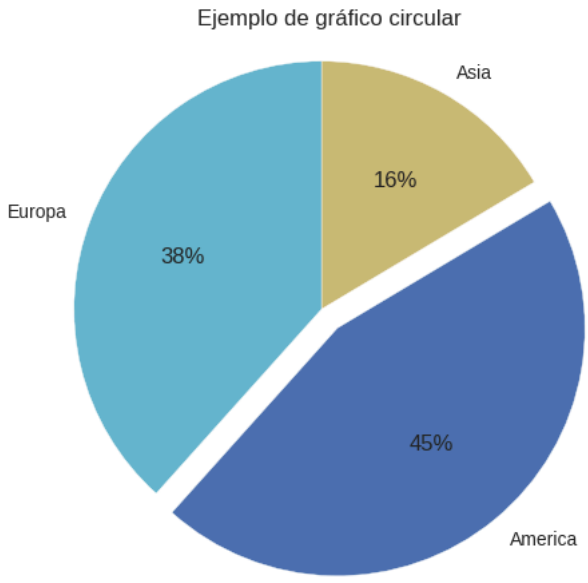
Gráficos circulares (Pie Charts)

```
import matplotlib.pyplot as plt

labels = 'Europa', 'America', 'Asia'
sections = [56, 66, 24]
colors = ['c', 'b', 'y']

plt.pie(sections, labels=labels, colors=colors,
        startangle=90,
        explode = (0, 0.1, 0),
        autopct = '%1.0f%%')

plt.axis('equal') #
plt.title('Ejemplo de gráfico circular')
plt.show()
```



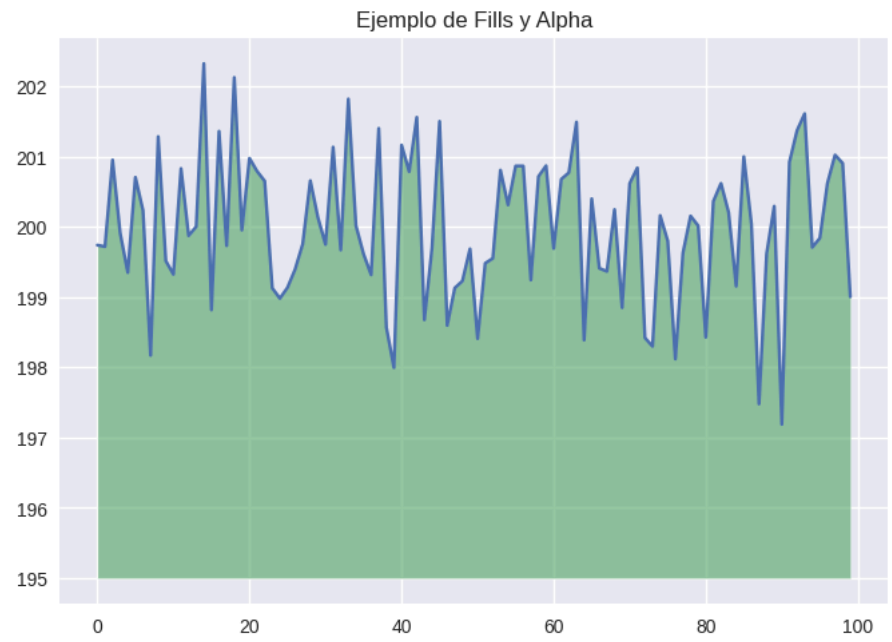
fill_between y alpha

```
import matplotlib.pyplot as plt
import numpy as np

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)

plt.title("Ejemplo de Fills y Alpha")
plt.show()
```



Sub trazado utilizando Subplot2grid

```
import matplotlib.pyplot as plt
import numpy as np

def random_plots():
  xs = []
  ys = []

  for i in range(20):
    x = i
    y = np.random.randint(10)

    xs.append(x)
    ys.append(y)

  return xs, ys

fig = plt.figure()
ax1 = plt.subplot2grid((5, 2), (0, 0), rowspan=1, colspan=2)
ax2 = plt.subplot2grid((5, 2), (1, 0), rowspan=3, colspan=2)
ax3 = plt.subplot2grid((5, 2), (4, 0), rowspan=1, colspan=1)
ax4 = plt.subplot2grid((5, 2), (4, 1), rowspan=1, colspan=1)

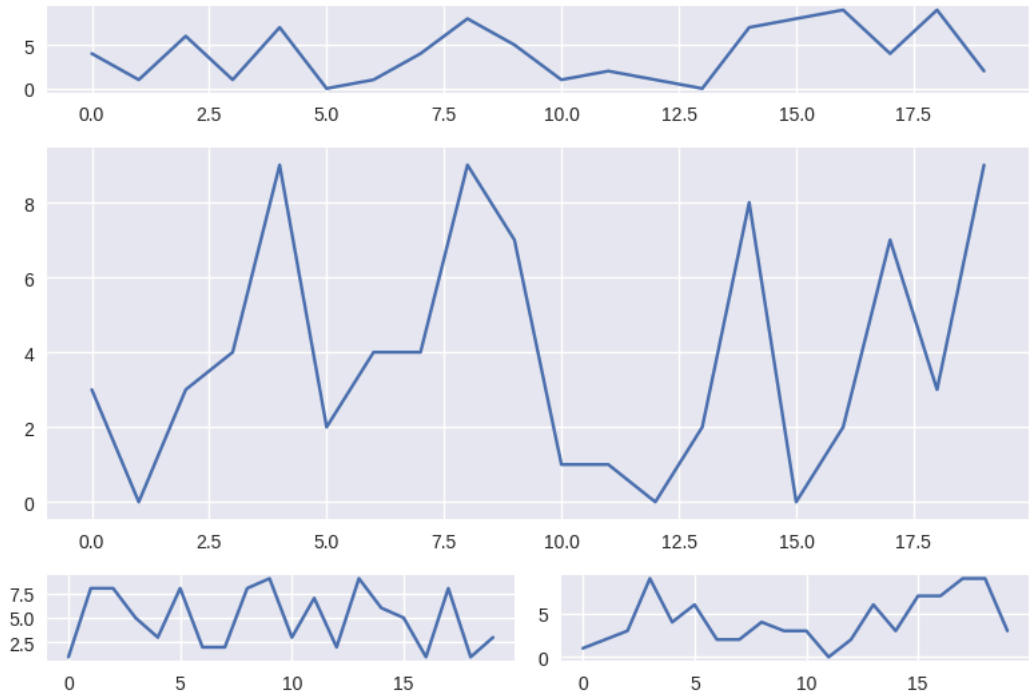
x, y = random_plots()
ax1.plot(x, y)

x, y = random_plots()
ax2.plot(x, y)

x, y = random_plots()
ax3.plot(x, y)

x, y = random_plots()
ax4.plot(x, y)
```

```
plt.tight_layout()
plt.show()
```



Plot styles

Colaboratory charts use Seaborn's custom styling by default. To customize styling further please see the matplotlib docs.

3D Graphs

3D Scatter Plots

```
import matplotlib.pyplot as plt
import numpy as np

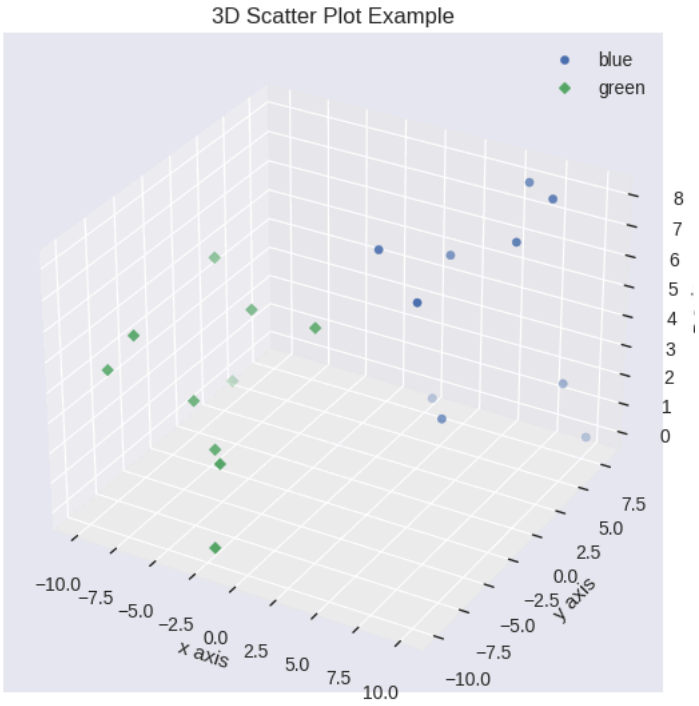
fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')

x1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y1 = np.random.randint(10, size=10)
z1 = np.random.randint(10, size=10)

x2 = [-1, -2, -3, -4, -5, -6, -7, -8, -9, -10]
y2 = np.random.randint(-10, 0, size=10)
z2 = np.random.randint(10, size=10)

ax.scatter(x1, y1, z1, c='b', marker='o', label='blue')
ax.scatter(x2, y2, z2, c='g', marker='D', label='green')

ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
ax.set_zlabel('z axis')
plt.title("3D Scatter Plot Example")
plt.legend()
plt.tight_layout()
plt.show()
```



3D Bar Plots

```
import matplotlib.pyplot as plt
import numpy as np

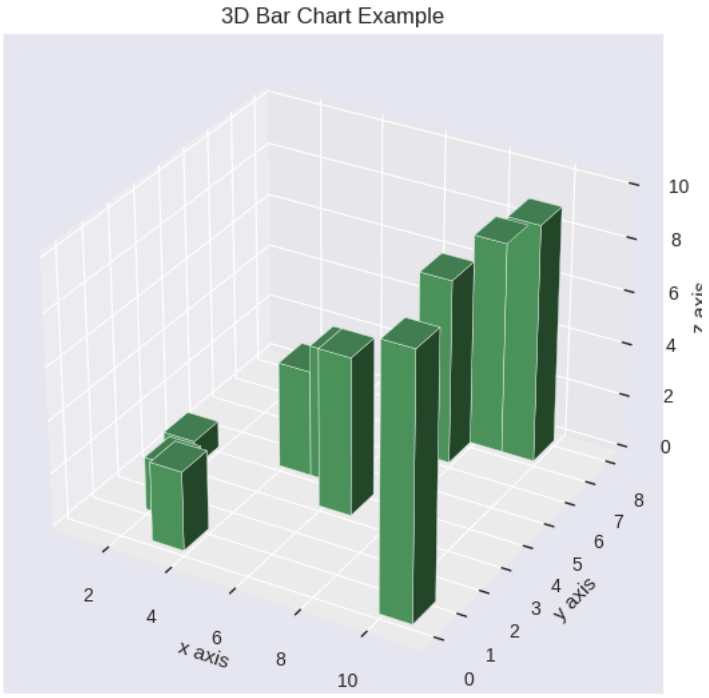
fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = np.random.randint(10, size=10)
z = np.zeros(10)

dx = np.ones(10)
dy = np.ones(10)
dz = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

ax.bar3d(x, y, z, dx, dy, dz, color='g')

ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
ax.set_zlabel('z axis')
plt.title("3D Bar Chart Example")
plt.tight_layout()
plt.show()
```



Wireframe Plots

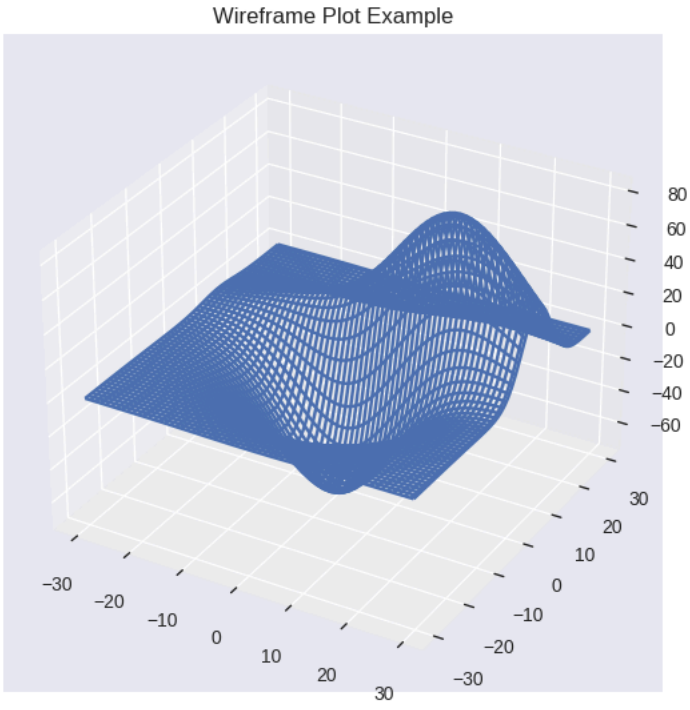
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import axes3d

fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')

x, y, z = axes3d.get_test_data()

ax.plot_wireframe(x, y, z, rstride = 2, cstride = 2)

plt.title("Wireframe Plot Example")
plt.tight_layout()
plt.show()
```



Seaborn

There are several libraries layered on top of Matplotlib that you can use in Colab. One that is worth highlighting is Seaborn:

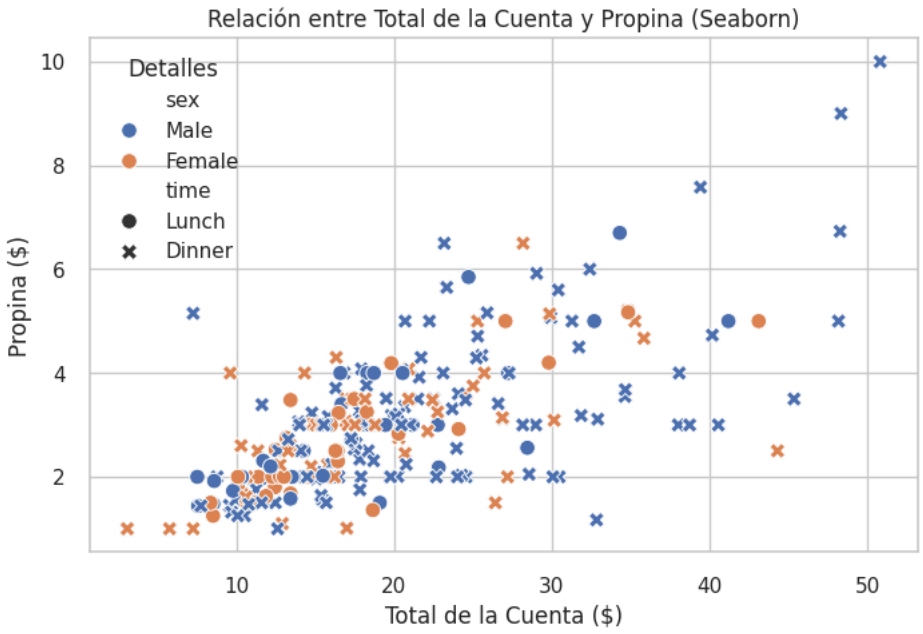
```
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración de estilo
sns.set_theme(style="whitegrid")

# Cargamos un dataset clásico incluido en seaborn (tips/propinas)
tips = sns.load_dataset("tips")

# Generar un gráfico de dispersión con línea de regresión y categorías por sexo
plt.figure(figsize=(8, 5))
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="sex", style="time", s=70)

plt.title("Relación entre Total de la Cuenta y Propina (Seaborn)")
plt.xlabel("Total de la Cuenta ($)")
plt.ylabel("Propina ($)")
plt.legend(title="Detalles")
plt.show()
```

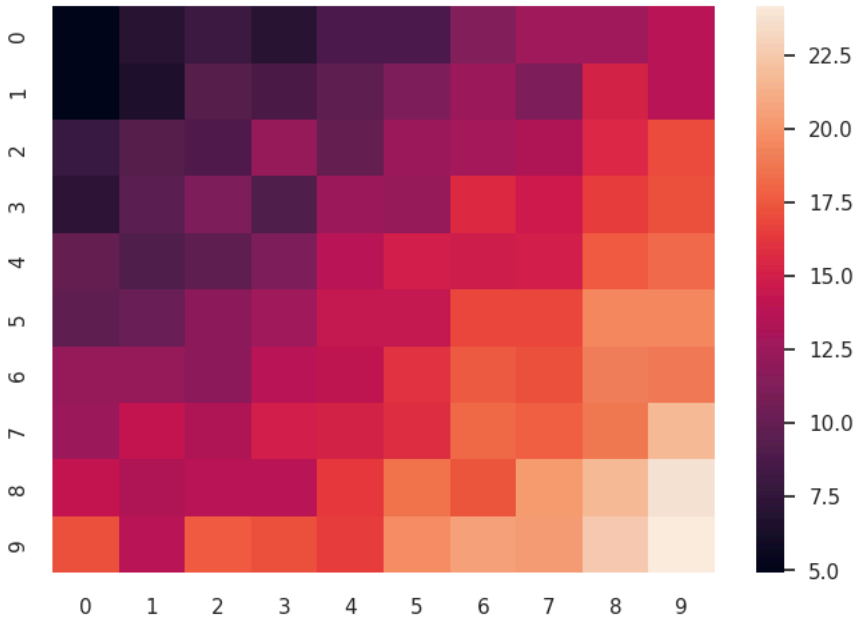


That's a simple scatterplot with a nice regression line fit to it, all with just one call to Seaborn's regplot.

Here's a Seaborn heatmap:

```
import matplotlib.pyplot as plt
import numpy as np

# Make a 10 x 10 heatmap of some random data
side_length = 10
# Start with a 10 x 10 matrix with values randomized around 5
data = 5 + np.random.randn(side_length, side_length)
# The next two lines make the values larger as we get closer to (9, 9)
data += np.arange(side_length)
data += np.reshape(np.arange(side_length), (side_length, 1))
# Generate the heatmap
sns.heatmap(data)
plt.show()
```



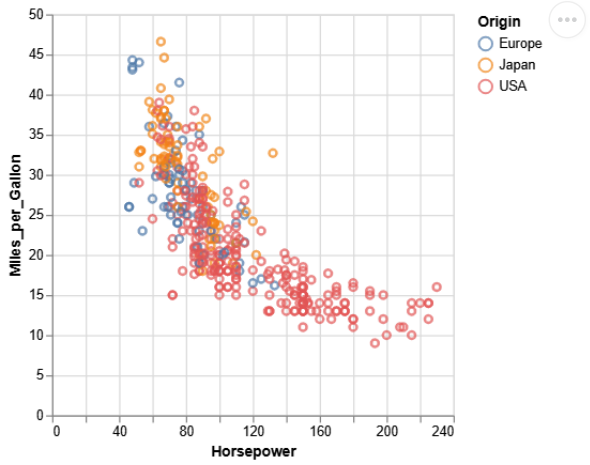
Altair

Altair is a declarative visualization library for creating interactive visualizations in Python, and is installed and enabled in Colab by default.

For example, here is an interactive scatter plot:

```
import altair as alt
from vega_datasets import data
cars = data.cars()

alt.Chart(cars).mark_point().encode(
    x='Horsepower',
    y='Miles_per_Gallon',
    color='Origin',
).interactive()
```



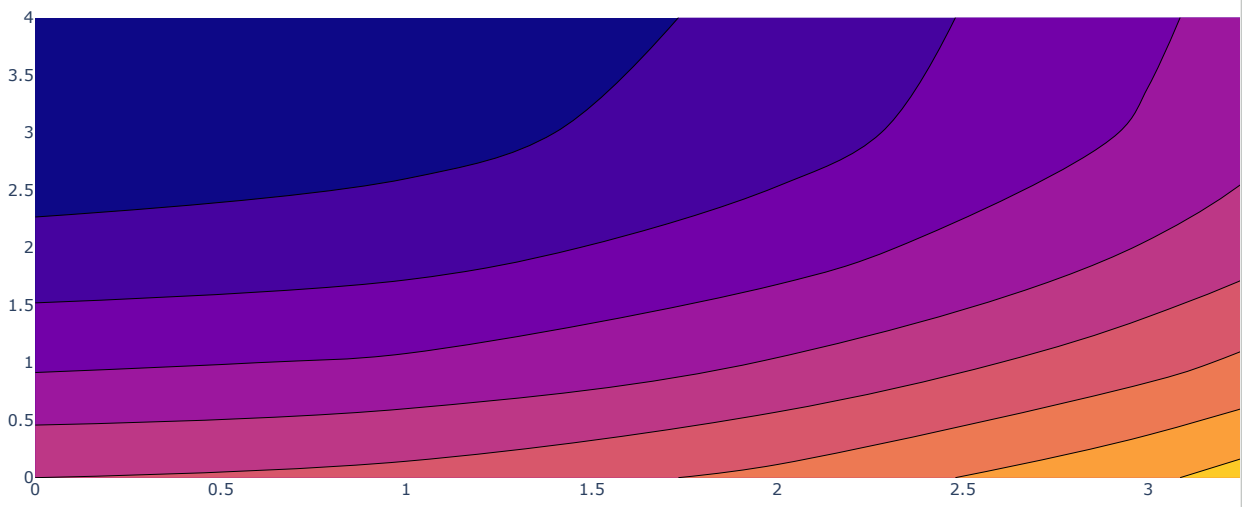
For more examples of Altair plots, see the Altair snippets notebook or the external Altair Example Gallery.

Plotly

Sample

```
from plotly.offline import iplot
import plotly.graph_objs as go

data = [
    go.Contour(
        z=[[10, 10.625, 12.5, 15.625, 20],
           [5.625, 6.25, 8.125, 11.25, 15.625],
           [2.5, 3.125, 5., 8.125, 12.5],
           [0.625, 1.25, 3.125, 6.25, 10.625],
           [0, 0.625, 2.5, 5.625, 10]]
    )
]
iplot(data)
```



Bokeh

Sample

```
import numpy as np
from bokeh.plotting import figure, show
from bokeh.io import output_notebook

# Call once to configure Bokeh to display plots inline in the notebook.
output_notebook()

N = 4000
x = np.random.random(size=N) * 100
y = np.random.random(size=N) * 100
radii = np.random.random(size=N) * 1.5
colors = ["#%02x%02x%02x" % (r, g, 150) for r, g in zip(np.floor(50+2*x).astype(int), np.floor(30+2*y).astype(int))]

p = figure()
p.circle(x, y, radius=radii, fill_color=colors, fill_alpha=0.6, line_color=None)
show(p)
```

...

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Definir los datos para el eje X
# Usamos np.linspace para generar 1000 puntos suaves entre -2 y 2
x = np.linspace(-2, 2, 1000)

# 2. Definir las funciones matemáticas para el corazón
# Función para la parte superior
y_superior = np.sqrt(1 - (np.abs(x) - 1)**2)
# Función para la parte inferior
y_inferior = -3 * np.sqrt(1 - (np.abs(x) / 2)**0.5)

# 3. Configurar la gráfica con un estilo bonito
plt.style.use('seaborn-v0_8') # Usamos el estilo que corregimos antes
fig, ax = plt.subplots(figsize=(8, 8))

# 4. Graficar ambas partes
# Usamos ax.plot para líneas suaves y ax.fill_between para rellenar de color
ax.fill_between(x, y_superior, y_inferior, color='red', alpha=0.7)
ax.plot(x, y_superior, color='firebrick', linewidth=2)
ax.plot(x, y_inferior, color='firebrick', linewidth=2)

# 5. Personalizar para que se vea estético
ax.set_title("¡Un Corazón Hecho con Matemáticas!", fontsize=16)
ax.set_xlim(-2.5, 2.5)
ax.set_ylim(-3.5, 1.5)
ax.axis('off') # Ocultamos los ejes para que solo se vea el corazón

# 6. ORDEN CORRECTO: Guardar antes de mostrar
plt.savefig('corazon_matematico.png', bbox_inches='tight')
plt.show()

print("¡El archivo 'corazon_matematico.png' ha sido generado!")
```




```
¡El archivo 'corazon_matematico.png' ha sido generado!
```

Desafio

- Importa la librería Pandas a tu proyecto.
- Genera un diccionario con al menos 10 elementos, donde las estructuras llave-valor correspondan a diferentes ciudades de México con su respectiva densidad de población (ejemplo: Guadalajara: 1,521,741).
- Genera una Series a partir del diccionario anterior.
- Incluye tres nuevos índices dentro de la Series, correspondientes a otras ciudades de México, y verifica que Pandas autocompletó los campos con los valores “NaN” para mantener el ajuste del arreglo.
- Finalmente, elige alguna de las herramientas de visualización vistas en clase y realiza la gráfica de los datos.

Solución Propuesta

Librerías

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Definición de la serie de población (en millones de habitantes aproximados)
datos_poblacion = {
    'Mérida': 1.0,
    'Toluca': 0.9,
    'Querétaro': 1.1,
    'Ciudad Juárez': 1.5,
    'Puebla': 1.7,
    'León': 1.7,
    'Tijuana': 1.9,
    'Monterrey': 5.3,
    'Guadalajara': 5.2,
```

```
    'Ciudad de México': 9.2
}
```

```
# Genera una Serie a partir del diccionario
serie_poblacion = pd.Series(datos_poblacion)
serie_ordenada = serie_poblacion.sort_values()
```

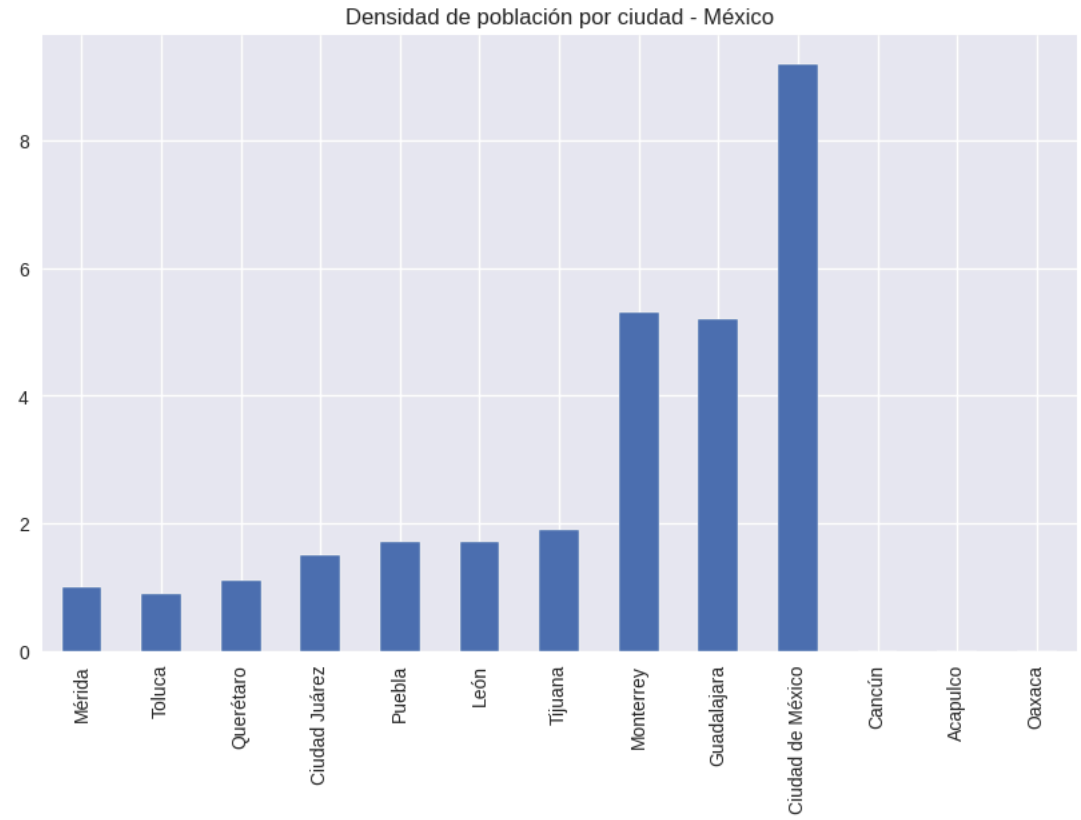
```
# Agrega tres nuevas ciudades con valores NaN
serie_poblacion['Cancún'] = pd.NA
serie_poblacion['Acapulco'] = pd.NA
serie_poblacion['Oaxaca'] = pd.NA

# Verifica la Serie con las nuevas ciudades y NaN
print(serie_poblacion)
```

```
Mérida              1.0
Toluca              0.9
Querétaro           1.1
Ciudad Juárez       1.5
Puebla              1.7
León                1.7
Tijuana             1.9
Monterrey           5.3
Guadalajara         5.2
Ciudad de México    9.2
Cancún              NaN
Acapulco            NaN
Oaxaca              NaN
dtype: float64
```

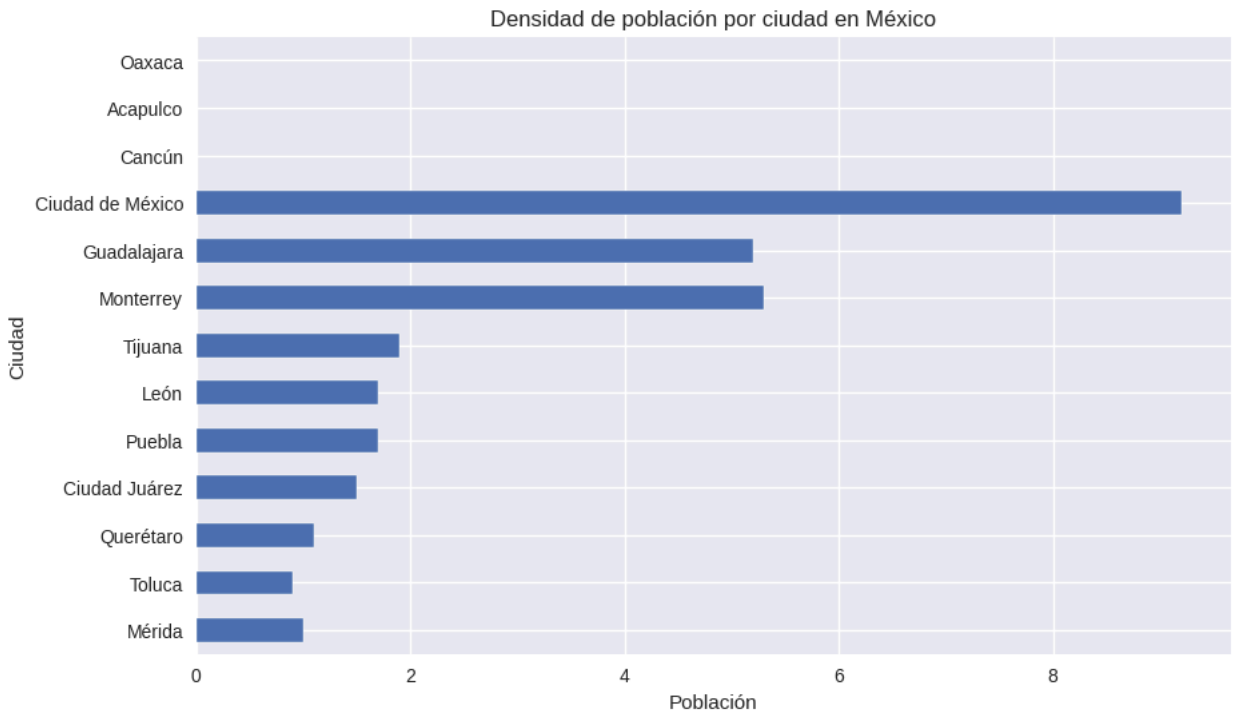
```
# Realiza la gráfica de los datos
serie_poblacion.plot(kind='bar', figsize=(10, 6), title='Densidad de población por ciudad - México')
```

```
<Axes: title={'center': 'Densidad de población por ciudad - México'}>
```

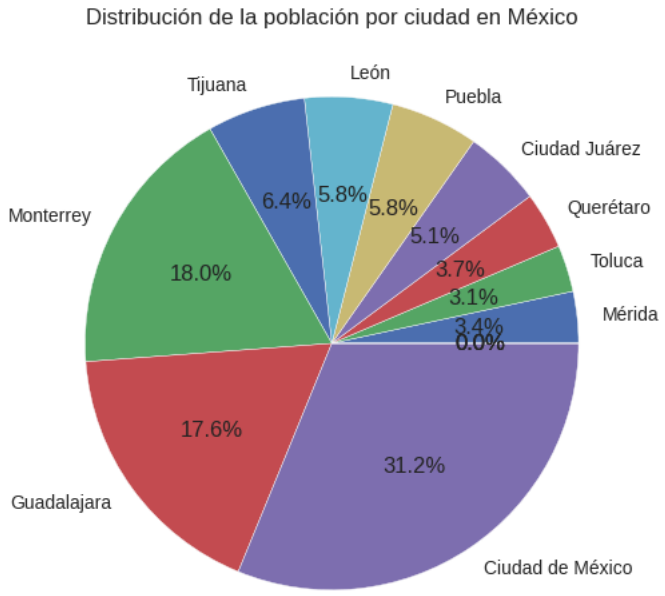



```
# Gráfico de barras horizontales
plt.figure(figsize=(10, 6))
serie_poblacion.plot(kind='barh')
plt.title('Densidad de población por ciudad en México')
plt.xlabel('Población')
plt.ylabel('Ciudad')
```

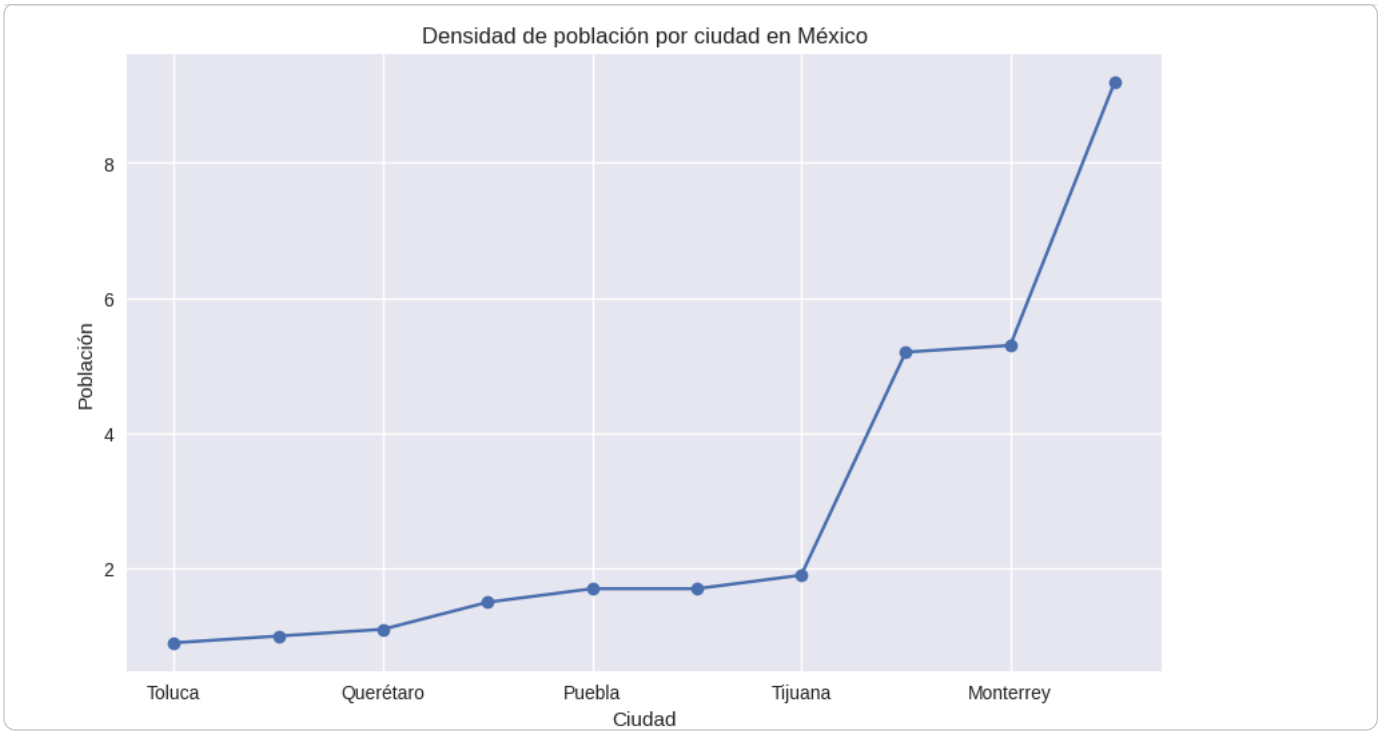
```
plt.ylabel('Ciudad')
plt.show()
```




```
# Gráfico de pastel
plt.figure(figsize=(10, 6))
serie_poblacion.plot(kind='pie', autopct='%1.1f%%')
plt.title('Distribución de la población por ciudad en México')
plt.ylabel('')
plt.show()
```




```
# Gráfico de líneas
plt.figure(figsize=(10, 6))
serie_poblacion.sort_values().plot(kind='line', marker='o')
plt.title('Densidad de población por ciudad en México')
plt.xlabel('Ciudad')
plt.ylabel('Población')
plt.grid(True)
plt.show()
```



Acumulativa

```
# Calcular la suma acumulativa de la población
serie_acumulativa = serie_poblacion.sort_values().cumsum()

# Gráfico acumulativo de líneas
plt.figure(figsize=(10, 6))
serie_acumulativa.plot(kind='line', marker='o')
plt.title('Población acumulativa por ciudad en México')
plt.xlabel('Ciudad')
plt.ylabel('Población acumulativa')
plt.grid(True)
plt.show()
```

